Operations Research Letters 51 (2023) 446–452

Contents lists available at ScienceDirect

Operations Research Letters

journal homepage: www.elsevier.com/locate/orl

On data reduction for dynamic vector bin packing

René van Bevern *, Andrey Melnikov, Pavel V. Smirnov, Oxana Yu. Tsidulko

Huawei Technologies Co., Ltd., Novosibirsk, Russian Federation

ARTICLE INFO

Article history:
Received 18 May 2022
Received in revised form 8 April 2023
Accepted 27 June 2023
Available online 4 July 2023

Keywords:
Approximation & heuristics
Parameterized complexity
Lossy kernelization
Resource allocation

ABSTRACT

We study a dynamic vector bin packing (DVBP) problem. We show hardness for shrinking arbitrary DVBP instances to size polynomial in the number of request types or in the maximal number of requests overlapping in time. We also present a simple polynomial-time data reduction algorithm that allows to recover $(1+\varepsilon)$-approximate solutions for arbitrary $\varepsilon > 0$. It shrinks instances from Microsoft Azure and Huawei Cloud by an order of magnitude for $\varepsilon = 0.02$.

1. Introduction

Motivated by applications to computer storage allocation, Coffman et al. [7] introduced the dynamic bin packing problem (DBP). Motivated by resource allocation problems in cloud data centers, we study the multi-dimensional generalization of the problem:

Problem 1.1 *(Dynamic vector bin packing (DVBP)).*
Input: A bin capacity $b \in \mathbb{N}^d$, *requests* $a^{(1)}, \dots, a^{(n)} \in \mathbb{N}^d$, request start times $s^{(1)}, \dots, s^{(n)} \in \mathbb{N}$, request end times $e^{(1)}, \dots, e^{(n)} \in \mathbb{N}$.
Find: A partition of $\{1, \dots, n\}$ into *bins* $B_1, \dots, B_k$ with minimum k such that, for any *time instant* $t \in \mathbb{N}$ and each bin B_j, it holds that (component-wise)

$$\sum_{\substack{i \in B_j \\ s^{(i)} \le t < e^{(i)}}} a^{(i)} \le b.$$

Definition 1.2 *(flavor, type, height).* Two requests $a^{(i)}$ and $a^{(j)}$ have the same *flavor* if $a^{(i)} = a^{(j)}$. They are of the same *type* if, additionally, $s^{(i)} = s^{(j)}$ and $e^{(i)} = e^{(j)}$.

The *height* h of an instance is the maximum number of requests active at the same time. We denote the number of flavors by ϕ, and the number of types by τ.

Example 1.3. Consider a pool of identical servers of capacity $b \in \mathbb{N}^2$, providing b_1 processors and b_2 units of memory. Customers make requests $a^{(i)} \in \mathbb{N}^2$ for virtual machines with $a_1^{(i)}$ processors and $a_2^{(i)}$ units of memory from time $s^{(i)}$ to $e^{(i)}$. The problem of satisfying all customer requests using a minimum number of servers is DVBP with $d = 2$. In practice, customers usually have the choice among a few virtual machine flavors.

In the virtual machine assignment scenario, the start and end times of requests are usually not known beforehand and the problem has to be solved online: on arrival, each request is immediately assigned to a bin, reassigning requests to other bins may be allowed or not [6]. Since, in the worst case, any online algorithm for DVBP will use $\Omega(d^{1-\varepsilon})$ times the optimal number of bins, cloud providers use various online heuristics tailored to their presumed input distributions [see, e.g., 20].

In order to empirically evaluate the quality of these heuristics, computing (close to) optimal solutions to the offline DVBP is desirable. Unfortunately, even when all requests have identical start and end times and $d = 2$, the best known polynomial-time approximation algorithm for DVBP yields an asymptotic approximation factor of $1.405 + \varepsilon$ [3]. Unless P = NP, asymptotic $(1+\varepsilon)$-approximations for arbitrarily small $\varepsilon > 0$ will require superpolynomial time [18,21].

Data reduction has proven to be a powerful way to cope with superpolynomial problem complexity [1,2].

Our results and outline of this work We study the potential for polynomial-time data reduction for DVBP.

In Section 2, we introduce basic tools and notation. In Section 3, we show hardness results for data reduction. In Section 4,

* Corresponding author.
E-mail addresses: rene.van.bevern@huawei.com (R. van Bevern), melnikov.andrey@huawei.com (A. Melnikov), smirnov.pavel2@huawei.com (P.V. Smirnov), tsidulko.oksana@huawei.com (O.Yu. Tsidulko).

https://doi.org/10.1016/j.orl.2023.06.005
0167-6377/

we present exact data reduction, and in Section 5 — data reduction that guarantees recoverability of $(1+\varepsilon)$-approximate solutions for any $\varepsilon > 0$.

Due to our hardness results, we cannot prove bounds on the data reduction effect. For this reason, the effect is evaluated experimentally in Section 6. We were able to shrink the number of request types in real-world instances to about 2% of their initial number, and the number of requests — to 8% on one data set, and to 20% on another.

2. Preliminaries

2.1. Parameterized optimization and decision problems

Definition 2.1. A *decision problem* is a subset $\Pi \subseteq \Sigma^*$ for some finite alphabet Σ. The task is, given an *instance* $x \in \Sigma^*$, determining whether $x \in \Pi$. If $x \in \Pi$, then x is a *yes-instance*. Otherwise, it is a *no-instance*.

For optimization problems, we use the terminology of Garey and Johnson [13]. We will only consider *minimization problems* in our work.

Definition 2.2. A *combinatorial optimization problem* Π is a triple $\Pi = (D_\Pi, S_\Pi, m_\Pi)$, where

1. D_Π is a set of *instances*,
2. S_Π is a function assigning to each instance $I \in D_\Pi$ a finite set $S_\Pi(I)$ of *(feasible) solutions*, and
3. m_Π is a function assigning a *solution cost* $m_\Pi(I, \sigma)$ to each feasible solution $\sigma \in S_\Pi(I)$ of an instance $I \in D_\Pi$.

An *optimal solution* for an instance $I \in D_\Pi$ is a feasible solution $\sigma \in S_\Pi(I)$ minimizing $m_\Pi(I, \sigma)$. Its cost is denoted as $\mathrm{OPT}_\Pi(I)$, where we drop the subscript Π when the optimization problem is clear from context.

An *α-approximate solution* for an instance I of a combinatorial optimization problem Π is a feasible solution of cost at most $\alpha \cdot \mathrm{OPT}_\Pi(I)$.

Each optimization problem comes with a natural associated *decision version* and a *gap version*:

Definition 2.3. Let Π be a combinatorial optimization problem and let $\alpha \geq 1$.

By *α-gap Π* we denote the following decision problem: given a number r and an instance I of Π such that $\mathrm{OPT}(I) \leq r$ or $\mathrm{OPT}(I) > \alpha r$, it is required to decide whether $\mathrm{OPT}(I) \leq r$.

For $\alpha = 1$, we call α-gap Π simply the *decision version of* Π.

If α-gap Π is NP-hard, then Π cannot be better than α-approximated in polynomial-time, unless $\mathrm{P} = \mathrm{NP}$.

We use the parameterized complexity notation due to Flum and Grohe [11], as it equally well applicable to decision and optimization problems.

Definition 2.4. A *parameterization* is a polynomial-time computable mapping $\kappa\colon \Sigma^* \to \mathbb{N}$ of instances (of decision or optimization problems) to a *parameter*. For a (decision or optimization) problem Π and parameterization κ, the pair (Π, κ) is called a *parameterized (decision or optimization) problem*.

2.2. Data reduction with performance guarantees

A *data reduction* is a conversion of a problem input into a "similar" but smaller one. There are multiple ways to formalize the term. A well-known notion of data reduction with performance guarantees is kernelization [11]:

Definition 2.5. A *kernelization* for a parameterized decision problem (Π, κ) is a polynomial-time algorithm that maps any instance $x \in \Sigma^*$ to an instance $x' \in \Sigma^*$ such that

(i) $x \in \Pi \iff x' \in \Pi$, and
(ii) $|x'| \leq g(\kappa(x))$ for some computable function g.

We call x' the *problem kernel* and g its *size*.

We will also consider approximate data reduction [17]:

Definition 2.6. An *α-approximate preprocessing* for a parameterized optimization problem (Π, κ) consists of two algorithms:

(i) The first algorithm reduces an instance I of Π to an instance I' of Π in polynomial time,
(ii) The second algorithm turns any β-approximate solution for I' into an $\alpha\beta$-approximate solution for I in polynomial time.

An *α-approximate kernelization* is an α-approximate preprocessing such that $|I'| \leq g(\kappa(I))$ for some computable function $g\colon \mathbb{N} \to \mathbb{N}$. We call g the *size* of the *approximate kernel* I'.

2.3. Hardness of data reduction

To exclude problem kernels of polynomial size, we use *AND-compositions* [5].

Definition 2.7 *(AND-composition).* An equivalence relation over Σ^* is *polynomial* if

— there is an algorithm that decides $x \sim y$ in polynomial time for any two instances $x, y \in \Sigma^*$, and
— the number of equivalence classes of $\sim$ over any *finite* set $S \subseteq \Sigma^*$ is polynomial in $\max_{x \in S} |x|$.

A language $L \subseteq \Sigma^*$ *AND-composes* into a parameterized language (Π, κ) if there is a polynomial-time algorithm, called *AND-composition*, that, given s instances $x_1, \ldots, x_s$ that are equivalent under some polynomial equivalence relation, outputs an instance x^* such that

— $\kappa(x^*) \in \mathrm{poly}(\max_{i=1}^{p} |x_i| + \log s)$,
— $x^* \in \Pi$ if and only if $x_i \in L$ for all $i \in \{1, \ldots, s\}$.

Proposition 2.8 *(Drucker [9]). If an NP-hard language $L \subseteq \Sigma^*$ AND-composes into a parameterized problem (Π, κ), then there is no polynomial-size problem kernel for (Π, κ) unless coNP $\subseteq$ NP/poly.*

3. Classification results

In this section, we prove that, unless the polynomial-time hierarchy collapses, DVBP has no problem kernels of size polynomial in $h + \phi$. Our reduction gives good reason to conjecture that there are no α-approximate kernels for any $\alpha < 600/599$, either.

3.1. Existence of exponential-size kernels

Before proving the non-existence of problem kernels with size polynomial in $h+\phi$, we prove that, in principle, problem kernels (of exponential size) *do* exist.

A folklore result from parameterized complexity [12, Theorem 1.4] is that the following theorem gives a problem kernel of size $O(k^{2h}) \subseteq O(h^{2h})$ for DVBP.

Theorem 3.1. *The DVBP problem can be solved in $O(k^{2h} \cdot n + n \log n)$ time.*

Proof. In $O(n \log n)$ time, we transform the instance so that $s^{(i)}, e^{(i)} \in \{1, \dots, 2n\}$ for all $i \in \{1, \dots, n\}$ (see Proposition 4.2 in Section 4.2 for details). We then solve the problem using dynamic programming.

For each $t \in \{1, \dots, 2n\}$, let $S_t := \{j \mid s^{(j)} \leq t < e^{(j)}\}$ be the set of requests active at time t. For $t \in \{0, \dots, 2n\}$ and any partition $X_1 \uplus X_2 \uplus \dots \uplus X_k = S_t$ (here, $\uplus$ denotes disjoint unions), consider the predicate $T[t, X_1, X_2, \dots, X_k]$ that is true if and only if all requests j with $s^{(j)} \geq t$ can be packed into k bins so that requests X_i are packed into bin i for $i \in \{1, \dots, k\}$. Then $T[2n+1, X_1, X_2, \dots, X_k]$ is true for any choice of the sets $X_1 \uplus \dots \uplus X_k = S_{2n+1} = \emptyset$.

Moreover, $T[t, X_1, X_2, \dots, X_k]$ is true if and only if there is a partition $X'_1 \uplus X'_2 \uplus \dots \uplus X'_k = S_{t+1}$, such that $T[t+1, X'_1, X'_2, \dots, X'_k]$ is true and $X_i \cap S_t \cap S_{t+1} = X'_i \cap S_t \cap S_{t+1}$ for each $i \in \{1, \dots, k\}$. That is, the partitions $\{X_i\}_{i=1}^k$ and $\{X'_i\}_{i=1}^k$ put the requests in $S_t \cap S_{t+1}$ into the same bins.

Thus, each of the $O(k^h n)$ values of T can be computed in $O(k^h)$ time by iterating over all possible partitions $\{X'_i\}_{i=1}^k$, yielding a total running time of $O(k^{2h} n)$. The answer can be found in $T[0, \emptyset, \emptyset, \dots, \emptyset]$. □

3.2. Non-existence of polynomial-size kernels

We now prove that, unless the polynomial-time hierarchy collapses, DVBP has no problem kernels with size polynomial even in $h+\phi$. Indeed, not even 600/599-gap DVBP has such a kernel:

Theorem 3.2. *α-gap DVBP does not have a problem kernel of size polynomial in $h+\phi$, unless the polynomial hierarchy collapses, for any $\alpha \leq 600/599$.*

Note that the statement of Theorem 3.2 holds for *any* parameter bounded from above by $h+\phi$. Moreover, the theorem leads us to the following conjecture.

Conjecture 3.3. *DVBP has no α-approximate problem kernels of size polynomial in $h+\phi$, for any $\alpha < 600/599$.*

Unfortunately, we cannot formally connect this conjecture, for example, to a collapse of the polynomial-time hierarchy, the main obstacle being the final open question in the work of Lokshtanov et al. [17].

In order to prove Theorem 3.2, we prove that the following problem AND-composes into DVBP.

Problem 3.4 *(2D vector bin packing (2D-VBP)).*
Input: Requests $a^{(1)}, \dots, a^{(n)} \in (0,1]^2$.
Find: A partition of $\{1, \dots, n\}$ into bins $B_1, \dots, B_k$ with minimum k such that, for each bin B_j, it holds that (component-wise)

$$\sum_{i \in B_j} a^{(i)} \leq 1.$$

Ray [18, Theorem 6] proved that 600/599-gap 2D-VBP does not have polynomial-time algorithms with asymptotic approximation ratio better than 600/599. In fact, he shows that 600/599-gap 2D-VBP is NP-hard. We show that NP-hardness also holds for the following problem variant, in which all numbers are integer and bounded by a polynomial in the number of items.

Problem 3.5 *(Poly-weight 2D-VBP).*
Input: Bin capacity $b \in \mathbb{N}^2$ and requests $a^{(1)}, \dots, a^{(n)} \in \mathbb{N}^2$, all bounded by a polynomial in n.
Find: A partition of $\{1, \dots, n\}$ into bins $B_1, \dots, B_k$ with minimum k such that, for each bin B_j, it holds that (component-wise)

$$\sum_{i \in B_j} a^{(i)} \leq b.$$

Lemma 3.6. *The 600/599-gap version of Problem 3.5 is NP-hard.*

Proof. The NP-hard maximum 3-dimensional matching problem (MAX-3-DM) is, given three sets $X = \{x_1, x_2 \dots, x_q\}$, $Y = \{y_1, y_2 \dots, y_q\}$, and $Z = \{z_1, z_2, \dots, z_q\}$, and a set of triples $T \subseteq X \times Y \times Z$, to find a set $T' \subseteq T$ of maximum cardinality such that each element of X, Y, and Z occurs in at most one triple in T'. Ray [18] showed a reduction of MAX-3-DM to 600/599-gap 2D-VBP. We modify his reduction so that it outputs instances of Problem 3.5 instead, without changing optimum number of bins.

To this end, we first briefly describe the reduction, omitting the correctness proof. Let $r := 64q$, and, for each $x_i \in X$ let $x'_i := ir + 1$, for each $y_i \in Y$ let $y'_i := ir^2 + 2$, for each $z_i \in Z$ let $z'_i := ir^3 + 4$, and for each triple $(x_i, y_j, z_k) \in T$, let $t'_{(i,j,k)} = r^4 - kr^3 - jr^2 - ir + 8$. Call the set of all these integers U'. The key feature of this construction is that the four numbers x'_i, y'_j, z'_k, and $t'_{(i,j,k)}$ sum up to exactly $b := r^4 + 15$. The final 2D-VBP instance consists of the requests

$$\left(\frac{1}{5} + \frac{a'}{5b}, \frac{3}{10} - \frac{a'}{5b}\right) \quad \text{for each } a' \in U'$$

and at most $|T| + 3q$ (the exact number is a technical detail) additional *dummy* requests of the form

$$\left(\frac{3}{5}, \frac{3}{5}\right).$$

The bin capacity, by definition of 2D-VBP, can be considered as $(1,1)$. The key observation here is that a bin can fit four requests if and only if the requests were built from the integers x'_i, y'_j, z'_k, and $t'_{(i,j,k)}$, or, in other words, if the requests correspond to some triple in T.

All requests are vectors of rational numbers whose denominators have a common multiple

$$D := 10 \cdot (r^4 + 15) = 10 \cdot ((64q)^4 + 15)$$

and whose numerators are all bounded by a polynomial of q. Multiplying all requests by D and setting a bin capacity of (D, D), we get an equivalent instance of Problem 3.5 in which all numbers are natural and bounded by a polynomial in q. Since the number n of requests in it is at least $3q$, all numbers are bounded polynomially in the number n of requests. Thus, Ray's result holds for Problem 3.5. □

Lemma 3.6 makes Problem 3.5 fundamentally different from standard Bin Packing, whose 3/2-gap version is NP-hard, but only weakly so — if the item sizes are bounded by a polynomial in the number of items, standard Bin Packing is polynomial-time solvable.

The polynomial bound on the request sizes plays a crucial role in bounding the number of request flavors ϕ in the following result:

Lemma 3.7. *The α-gap version of Problem 3.5 AND-composes into α-gap DVBP parameterized by $h+\phi$.*

Proof. Assume instances $I_1, I_2, \ldots, I_s$ of the α-gap version of Problem 3.5. Without loss of generality, we may assume that each of them consists of the same number n of requests, each is asking for the same number r of bins and each instance's bin size is b (since this is a polynomial equivalence relation). As a result, the entries of all vectors are bounded by $\text{poly}(n)$.

One now can create a DVBP instance I consisting of all requests of all instances I_i for $i \in \{1, \ldots, s\}$, assigning the requests of instance I_i the start time $2i-1$ and the end time $2i$. Obviously, the created DVBP instance I can fit into r bins if and only if each of the instances I_i can fit into r bins. Moreover, if at least one of the instances I_i requires more than r bins, then it requires at least αr bins, meaning that I requires at least αr bins.

Also note that the maximum number of requests active at any time in I is n and that the number of flavors in I is bounded from above by $\text{poly}(n)$. That is, $h + \phi \leq \text{poly}(n) \leq \text{poly}(\max_{i=1}^{s} |I_s|) + \log s$; we thus built a valid AND-composition. □

Finally, Theorem 3.2 now follows from Proposition 2.8 and Lemma 3.7 and the NP-hardness of the α-gap variant of Problem 3.5 for all $\alpha \leq 600/599$ (Lemma 3.6).

4. Exact data reduction

Due to the results in the previous section, we do not expect $(1+\varepsilon)$-approximate preprocessing for DVBP for arbitrarily small $\varepsilon > 0$ with provable *a priori size* bounds of the output. This, however, does not prevent us from designing data reduction algorithms that show good *a posterior* results in experiments.

In this section, we describe a simple data reduction rule that maintains optimality of solutions.

4.1. DVBP with multiplicities

Motivated by the following theorem, which we prove in this section, our first data reduction approach will focus on lowering the number τ of request types.

Theorem 4.1. *DVBP with k bins and τ request types is solvable in $2^{O(\tau k \log \tau k)} \cdot \text{poly}(n)$ time.*

To prove the theorem, we follow an approach of Fellows et al. [10] for deriving algorithms for packing problems parameterized by the "number of numbers". Specifically, we modify an ILP model for DVBP of Dell'Amico et al. [8, Section 3.1], replacing Boolean variables by integer variables so as to allow for requests with multiplicities. The number of variables in the ILP will be $O(\tau k)$, so that Theorem 4.1 immediately follows from a result of Lenstra [16] and Kannan [15].

To state the ILP, let $T := \max_{i=1}^{n} e^{(n)}$ be the latest time that any request ends, $S_t := \{j \mid s^{(j)} \leq t < e^{(j)}\}$ be the requests active at time $t \in \{0, \ldots, T\}$, and let $n^{(i)}$ be the number of requests of type i, $1 \leq i \leq \tau$. The model involves the following variables:

- x_{ij} for $1 \leq i \leq \tau$ and $1 \leq j \leq k$ is the number of requests of type i packed into bin j.
- y_j for $1 \leq j \leq k$ equals one if bin j is used during at least one time instant and zero otherwise.

Then, DVBP is modeled using the following ILP.

$$\min \sum_{j=1}^{k} y_j \qquad \text{s.t.}$$

$$\sum_{j=1}^{k} x_{ij} = n^{(i)} \qquad 1 \leq i \leq \tau, \tag{1}$$

$$x_{ij} \leq n^{(i)} y_j, \qquad 1 \leq i \leq \tau, 1 \leq j \leq k, \tag{2}$$

$$\sum_{i \in S_t} x_{ij} a^{(i)} \leq y_j b, \qquad 1 \leq t \leq T, 1 \leq j \leq \tau \tag{3}$$

$$y_j \in \{0, 1\}, \qquad 1 \leq j \leq k$$

$$x_{ij} \in \mathbb{N}, \qquad 1 \leq i \leq \tau, 1 \leq j \leq j.$$

Herein, constraint (1) ensures that each request is assigned to some bin, constraint (2) makes sure that bin j is counted as "used" whenever some request i is assigned to it, and constraint (3) ensures that the bin capacity b is not exceeded at any time instant t.

4.2. Time compression

The number of variables in the above ILP is $O(\tau k)$, whereas the number of constraints is $O(Tr + \tau k)$. It is therefore desirable to reduce τ and T. Since requests of the same flavor and same start and end times are of the same type, both reduction of τ and T is achieved by the following simple approach.

Proposition 4.2 *([4]). In $O(n \log n)$ time, the start and end points of all requests can be moved to an interval $\{1, \ldots, T\}$ with minimum T maintaining all pairwise intersections of request intervals.*

In Section 6, we will see that this data reduction alone reduces the number of request types by about one third in real problem instances.

5. $(1+\varepsilon)$-approximate data reduction

In this section, we show a $(1+\varepsilon)$-approximate data reduction algorithm for DVBP, that is, the cost of solutions may increase at most $(1+\varepsilon)$ times.

In view of the hardness results in Section 3, we do not expect to prove meaningful a priori size bounds of the output. Indeed, as we will see, maximizing the data reduction effect of the algorithm is an NP-hard subproblem itself. Therefore, we first describe the high-level algorithm and then, in subsequent subsections, describe heuristics for its effective execution. The data reduction effect will be empirically evaluated in Section 6.

5.1. A $(1+\varepsilon)$-approximate data reduction rule

Let L be a lower bound on the number of bins required to accommodate all requests. For example, assuming that all request start and end times are in $\{1, \ldots, 2n\}$ by Proposition 4.2, one can take

$$S_t := \{j \mid s^{(j)} \leq t < e^{(j)}\},$$

$$L := \max_{t \in \{1, \ldots, 2n\}} \max_{i \in \{1, \ldots, d\}} \sum_{j \in S_t} \frac{a_i^{(j)}}{b_i}. \tag{4}$$

Reduction Rule 5.1. Delete an arbitrary set of requests that can be packed into $\lfloor \varepsilon L \rfloor$ bins.

Algorithm 1: Greedy packing into one bin.

Input : A bin capacity $b \in \mathbb{N}^d$,
requests $a^{(1)}, \dots, a^{(n)} \in \mathbb{N}^d$ with
start times $s^{(1)}, \dots, s^{(n)} \in \{1, \dots, T\}$,
end times $e^{(1)}, \dots, e^{(n)} \in \{1, \dots, T\}$, and
priorities $f : \{1, \dots, n\} \mapsto \mathbb{Q}$.
Result: Indices of packed requests.

```
A ← ∅;
S ← [b, b, ..., b] of size T;
for a^(i) in non-increasing order of f(i) do
    if a^(i) ≤ S[t] for t ∈ {s^(i), ..., e^(i) − 1} then
        A ← A ∪ {i};
        for t ∈ {s^(i), ..., e^(i) − 1} do
            S[t] ← S[t] − a^(i);
return A;
```

Theorem 5.2. *Reduction Rule 5.1 is a $(1+\varepsilon)$-preprocessing for DVBP.*

Proof. Let I be the DVBP instance before and I' be the DVBP instance after applying Reduction Rule 5.1. If all requests of I can be packed into k bins, then the requests of I' also can. Thus $\mathrm{OPT}(I') \leq \mathrm{OPT}(I)$. If all requests of I' can be packed into k bins, then the requests of I can be packed into $k + \lfloor \varepsilon L \rfloor$ bins. Thus, any α-approximate solution for I' can be turned into an $\alpha(1+\varepsilon)$-approximate solution for I since

$$\begin{aligned}\alpha\mathrm{OPT}(I') + \varepsilon L &\leq \alpha\mathrm{OPT}(I) + \varepsilon\mathrm{OPT}(I)\\ &\leq \alpha(1+\varepsilon)\mathrm{OPT}(I). \quad \square\end{aligned}$$

In order to delete the maximum number of requests using Reduction Rule 5.1, one has to pack a maximum number of requests into $\lfloor \varepsilon L \rfloor$ bins, which is an NP-hard task. One possible implementation of Reduction Rule 5.1 is solving this task heuristically. Indeed, our experiments in Section 6 show that, solving the task exactly would not significantly increase the effect of Reduction Rule 5.1 on our real-world data.

5.2. Greedily packing εL bins

In order to pack εL bins, we repeatedly apply a greedy algorithm for packing one bin.

In order to pack one bin, we assign each request a priority, then iterate over all not yet packed requests by non-increasing priority and, if the request still fits into the bin, we pack it. In detail, the procedure is described in Algorithm 1, where the array S maintains the available bin space at each time instant and A is the set of requests packed into the bin.

The priorities are chosen as follows. Since the goal of the algorithm is to pack as many requests as possible, it makes sense to first pack those requests that require less space and block less time instants. A convenient value to use as an indicator of a request's small size is the number of requests of the same flavor as $a^{(i)}$ that can fit into one bin. Thus, a canonical priority assignment to each request i is

$$\alpha(i) := \min_{j \in \{1,\dots,d\}} \left\lfloor \frac{b_j}{a^{(i)}_j} \right\rfloor.$$

From $\alpha(i)$, we also derive the following priorities:

$$f_1(i) := \alpha(i) + \frac{1}{2(e^{(i)} - s^{(i)})},$$

which uses the time span as a tie-breaker for $\alpha(i)$ and places the shorter spanned requests earlier; and

Algorithm 2: Computation of upper bound U.

Input : A bin capacity $b \in \mathbb{N}^d$,
requests $a^{(1)}, \dots, a^{(n)} \in \mathbb{N}^d$ with
start times $s^{(1)}, \dots, s^{(n)} \in \{1, \dots, T\}$,
end times $e^{(1)}, \dots, e^{(n)} \in \{1, \dots, T\}$, and
the number of bins $k \in \mathbb{N}$.
Result: An upper bound U on the number of requests fitting into k bins.

```
1 S ← size-d-array of empty lists;
2 for any request type (a^(*), s^(*), e^(*)) of any multiplicity m do
3     p ← min{m} ∪ { k · ⌊ b_j / a^(*)_j ⌋ | j ∈ {1, ..., d} };
4     for j ∈ {1, ..., d} do
5         append p times a^(*)_j to S[j];
6 for j ∈ {1, ..., d} do
7     sort S[j] in non-decreasing order;
8     U_j ← max{ q ≤ n | Σ_{t=1}^{q} S[j][t] ≤ b_j };
9 return min{U_j | j ∈ {1, ..., d}};
```

$$f_2(i) := \alpha(i) \cdot \frac{T}{e^{(i)} - s^{(i)}},$$

which is an upper bound on the number of requests of the same flavor and life time as $a^{(i)}$ that can be placed over all time instants.

We found that f_2 works the best among α, f_1, f_2, which we evaluated as follows.

5.3. Quality estimation of greedy packing

In the following, we describe how to measure the effectivity of our greedy packing algorithm, thus ultimately answering the question of how effectively we use Reduction Rule 5.1 and whether the effect of Reduction Rule 5.1 can be significantly increased by solving the problem of packing into εL bins in a more sophisticated way, or even optimally.

In order to estimate the effect of Reduction Rule 5.1 in comparison to its potential effect, we use two *utilization* values comparing the number n of requests in the input instance to the number n' of requests in the output instance (that is, $n - n'$ requests are deleted by Reduction Rule 5.1). The first value is

$$R := \frac{n - n'}{U} \leq 1, \tag{5}$$

where U is an upper bound on the number of requests removable by Reduction Rule 5.1. The closer R is to 1, the closer the data reduction effect is to the maximum possible. The second value is

$$K := \frac{n'}{n - U} \geq 1, \tag{6}$$

where $n - U$ is a lower bound on the number of remaining requests n'. Low values of K mean that the size of the reduced instance is close to what it could be if we packed the $\lfloor \varepsilon L \rfloor$ bins optimally.

After reducing an instance, we can evaluate the effect of Reduction Rule 5.1 quantified by R and K, using the upper bound U. One way to obtain U is to apply Algorithm 2 with $k = \lfloor \varepsilon L \rfloor$; it works as follows.

In line 2, it iterates over all request types, and, for each request type and its multiplicity m, in line 3, computes the maximum number p of requests of this type that fit into k bins. In line 5, for each $j \in \{1, \dots, d\}$, it builds a list S_j containing j-th components of this request type p times, because the other $m - p$ requests of the same type will not fit into the bins anyway. In line 8, for each $j \in \{1, \dots, d\}$, it computes an upper bound U_j by greedily packing the requests with the smallest value in the j-th component first,

Table 1
Data reduction effect of Proposition 4.2 and Reduction Rule 5.1 with $\varepsilon = 0.05$. Herein, L is the lower bound (4), T is the number of time instants, n is the number of requests, τ is the number of types, R and K are the utilization measures (5) and (6), respectively.

	Data set	L	T	n	τ	R	K
Huawei	initial	814	152 366	125 430	111 050		
	after Proposition 4.2		29 347	125 430	78 010		
	after Reduction Rule 5.1		356	10 079	1 798	0.969	1.576
Azure	initial	116864	4 650 911	5 559 800	3 792 136		
	after Proposition 4.2		695 408	5 559 800	2 271 582		
	after Reduction Rule 5.1		20 226	1 069 118	81 357	0.946	1.319

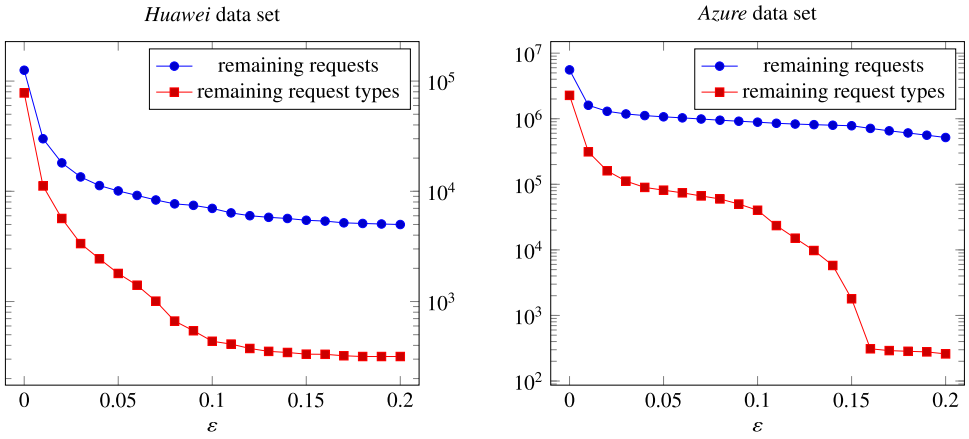

Fig. 1. Data reduction effect of Proposition 4.2 and Reduction Rule 5.1 for $\varepsilon \in [0, 0.2]$ with step size 0.01.

ignoring the other components. Finally, in line 8, the smallest of the upper bounds is returned.

6. Experiments

We present an experimental evaluation of the effect of our data reduction on real-world instances for a virtual machine scheduling problem (Example 1.3).

Data sets We evaluated our data reduction approach on six data sets with similar results. However, since four of the data sets are confidential, in detail we can present results here only for two openly available data sets.

The first one is "Huawei-East-1" released by Huawei [19].[1] We denote this data set *Huawei*. In this data set, $d = 2$, that is, each request has two resource requirements: the number of requested CPU cores and the amount of allocated RAM. As suggested by its authors, we simulated CPU sharing for small virtual machines (with flavors 2U4G, 4U8G, 8U16G, 1U2G, 4U16G, 1U1G, 2U8G, 8U32G and 1U4G, where machine $XUYG$ has X CPU cores and Y GB of RAM) by dividing their CPU demand by 3. We set the bin size to 40 CPU cores and 90 GB of RAM, following the data set authors' experimental setup. All arithmetic calculations for this data set are carried out with 64-bit integers.

The second data set, called *Azure*, is released by Microsoft and is called "Azure Traces for Packing 2020" [14].[2] This data set has $d = 5$, where each request has requirements on the number of CPU cores, amount of RAM, HDD and SSD space, and network bandwidth. In the *Azure* dataset, each resource requirement is a fractional number in $[0, 1]$ equal to the fraction of the available resource on the servers, so the bin size is set to $(1, 1, 1, 1, 1)$. All computations for this data set are done in double precision floating point arithmetic.

[1] https://github.com/huaweicloud/VM-placement-dataset.
[2] https://github.com/Azure/AzurePublicDataset.

Reduction effect In this section, we present experimental results of the $(1 + \varepsilon)$-approximate Reduction Rule 5.1. We perform Proposition 4.2 before Reduction Rule 5.1 and also during execution of Reduction Rule 5.1 after each packed and deleted bin.

Allowing an increase in the number of servers in an optimal solution by no more than 5%, we perform Reduction Rule 5.1 with $\varepsilon = 0.05$. Table 1 shows the results. When computing the optimal solution lower bound L according to (4), Reduction Rule 5.1 tries to pack into $\lfloor \varepsilon L \rfloor = 40$ bins for *Huawei* and into $\lfloor \varepsilon L \rfloor = 5843$ bins for *Azure*.

The data reduction given by Proposition 4.2 reduces the number of time instants by about 6 times. It does not change the number of requests, yet reduces the number of request types by about one third, since after time compression, many requests of the same flavor have coinciding start and end times and are thus of the same type.

After Reduction Rule 5.1, the number of requests is decreased significantly: 5.2 times for *Azure* and 12.4 times for *Huawei*. And even more significant is the change in the number of request types, which, for each data set, fell about 50 times.

A natural question is whether the effect of Reduction Rule 5.1 can be increased by packing the $\lfloor \varepsilon L \rfloor$ bins for deletion better. The table presents the resource utilization measures R and K described in Section 5.3. The value of R shows that our implementation of Reduction Rule 5.1 removes at least 94% of all requests that it can remove in principle. However, judging by K, Reduction Rule 5.1 potentially leaves 57% more requests in *Huawei* than there have to be and 32% more requests in *Azure* than have to be. On the one hand, this means that there might still be a little room for improvement, yet certainly not by an order of magnitude. On the other hand, the number of requests that have to remain is only a lower bound. Thus, it might even be that the $\lfloor \varepsilon L \rfloor$ bins are already packed optimally.

Finally, the choice of ε allows for a trade-off of solution quality versus output size of the data reduction, illustrated in Fig. 1. In both cases, we can observe data reduction by an order of magni-

tude for $\varepsilon = 0.02$. The number of request types shrinks by another order of magnitude for $\varepsilon \in [0.08, 0.12]$.

Good candidates for compromise are $\varepsilon = 0.1$ on the *Huawei* data set and $\varepsilon = 0.16$ on the *Azure* data set, if that would allow for solving the problem and such an error is permissible.

7. Conclusion

We have studied the potential of data reduction with performance guarantees for DVBP. While we have shown obstacles for proving size bounds of instances after data reduction, we proved guarantees on the solution quality.

Despite our data reduction rules being very simple and coming without size bounds, they proved to be very effective in experiments with real data from Microsoft Azure and Huawei Cloud, shrinking problem instances by orders of magnitude.

Unfortunately, for two the two open sets, the number of request types after data reduction is still out of reach for algorithms with running time exponential in the number of request types, for example, for the ILP from Theorem 4.1.

Data availability

Links to the used data are provided in footnotes.

References

[1] F.N. Abu-Khzam, S. Lamm, M. Mnich, A. Noe, C. Schulz, D. Strash, Recent Advances in Practical Data Reduction, Algorithms for Big Data, LNCS, vol. 13201, Springer, 2022, pp. 97–133.

[2] T. Achterberg, R.E. Bixby, Z. Gu, E. Rothberg, D. Weninger, Presolve reductions in mixed integer programming, INFORMS J. Comput. 32 (2020) 473–506, https://doi.org/10.1287/ijoc.2018.0857.

[3] N. Bansal, M. Eliáš, A. Khan, Improved approximation for vector bin packing, in: SODA 2016, SIAM, 2016, pp. 1561–1579.

[4] R. van Bevern, M. Mnich, R. Niedermeier, M. Weller, Interval scheduling and colorful independent sets, J. Sched. 18 (2015) 449–469, https://doi.org/10.1007/s10951-014-0398-5.

[5] H.L. Bodlaender, B.M.P. Jansen, S. Kratsch, Kernelization lower bounds by cross-composition, SIAM J. Discrete Math. 28 (2014) 277–305, https://doi.org/10.1137/120880240.

[6] H.I. Christensen, A. Khan, S. Pokutta, P. Tetali, Approximation and online algorithms for multidimensional bin packing: a survey, Comput. Sci. Rev. 24 (2017) 63–79, https://doi.org/10.1016/j.cosrev.2016.12.001.

[7] E.G. Coffman, M.R. Garey, D.S. Johnson, Dynamic bin packing, SIAM J. Comput. 12 (1983) 227–258, https://doi.org/10.1137/0212014.

[8] M. Dell'Amico, F. Furini, M. Iori, A branch-and-price algorithm for the temporal bin packing problem, Comput. Oper. Res. 114 (104) (2020) 825, https://doi.org/10.1016/j.cor.2019.104825.

[9] A. Drucker, New limits to classical and quantum instance compression, in: FOCS 2012, 2012, pp. 609–618.

[10] M.R. Fellows, S. Gaspers, F.A. Rosamond, Parameterizing by the number of numbers, Theory Comput. Syst. 50 (2011) 675–693, https://doi.org/10.1007/s00224-011-9367-y.

[11] J. Flum, M. Grohe, Parameterized Complexity Theory, Texts in Theoretical Computer Science, An EATCS Series, Springer, 2006.

[12] F.V. Fomin, D. Lokshtanov, S. Saurabh, M. Zehavi, Kernelization, Cambridge University Press, 2019.

[13] M.R. Garey, D.S. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness, Freeman, New York, USA, 1979.

[14] O. Hadary, L. Marshall, I. Menache, A. Pan, E.E. Greeff, D. Dion, S. Dorminey, S. Joshi, Y. Chen, M. Russinovich, T. Moscibroda, Protean: VM allocation service at scale, in: OSDI 2020, USENIX Association, 2020, pp. 845–861.

[15] R. Kannan, Minkowski's convex body theorem and integer programming, Math. Oper. Res. 12 (1987) 415–440, https://doi.org/10.1287/moor.12.3.415.

[16] H.W. Lenstra, Integer programming with a fixed number of variables, Math. Oper. Res. 8 (1983) 538–548, https://doi.org/10.1287/moor.8.4.538.

[17] D. Lokshtanov, F. Panolan, M.S. Ramanujan, S. Saurabh, Lossy kernelization, in: STOC 2017, ACM, 2017, pp. 224–237.

[18] A. Ray, There is no APTAS for 2-dimensional vector bin packing: revisited, ArXiv https://doi.org/10.48550/arXiv.2104.13362, 2021.

[19] J. Sheng, S. Cai, H. Cui, W. Li, Y. Hua, B. Jin, W. Zhou, Y. Hu, L. Zhu, Q. Peng, H. Zha, X. Wang, VMAgent: a practical virtual machine scheduling platform, in: IJCAI 2022, 2022.

[20] J. Shi, J. Luo, F. Dong, J. Jin, J. Shen, Fast multi-resource allocation with patterns in large scale cloud data center, J. Comput. Sci. 26 (2018) 389–401, https://doi.org/10.1016/j.jocs.2017.05.005.

[21] G.J. Woeginger, There is no asymptotic PTAS for two-dimensional vector packing, Inf. Process. Lett. 64 (1997) 293–297, https://doi.org/10.1016/s0020-0190(97)00179-8.